

통관 스마트 검증 API 개발 가이드 (CI 활용기관)

1. 서비스 개요

제공되는 API를 통해 통관 스마트 검증 서비스를 사용할 수 있습니다. 본 문서는 이용기관이 API를 직접 연동·개발할 수 있도록 가이드를 제공합니다.

- 현재 API의 제공 버전은 v1.0이며, 향후 버전 추가나 변경 사항은 본 사이트를 통해 안내될 예정입니다.

1.1 목적

본 문서는 서비스 간 연동을 위한 표준 API를 정의하고, 이용기관이 통관 스마트 검증 서비스를 효율적으로 연동·개발할 수 있도록 가이드를 제공하는 것을 목적으로 합니다.

1.2 대상

본 문서는 통관 스마트 검증 서비스를 연동 및 구현을 수행하는 개발자 및 관련 기술 담당자를 대상으로 작성되었습니다.

1.3 권한 제어

NICE 통합 인증 서비스를 이용하기 위해서는 먼저 이용기관 서버의 Outbound IP와 전자상거래업체 부호를 NICE 담당자에게 전달하여 권한 등록을 요청 해야 합니다.

- IP는 D-Class 및 C-Class 대역으로 등록이 가능합니다.
- 클라우드 서버를 사용할 경우 IP 주소를 NAT 처리하여 단일 IP로 통신되도록 설정 부탁드립니다.

권한 등록이 완료되면 client_id와 client_secret가 생성되며, 해당 정보는 이용기관 담당자에게 이메일로 전달됩니다.

발급된 client_id와 client_secret은 서비스 API 호출 시 인증 토큰 발급에 사용되며 반드시 안전하게 보관해야 합니다.

용어	비고	예시
client_id	NICE로 부터 전달 받은 값	NI1a34567-cde1-234a-bc1a-1a234567890
client_secret	NICE로 부터 전달 받은 값	Q1w2E3r4T5y6U7i8O9p0AaBbCcDdEeFfGgHhIjKkLlMmNnOoPpQqRrSsTtUuVvWwXxYyZz12345678

1.4 API URL 목록

API 코드	API명	API URI	설명	비고
SMARTPCC_01	접근토큰요청	/cons/pccc/v1.0/auth/token	24시간 유효한 AccessToken을 받기위한 API	
SMARTPCC_10	개인통관고유부호 유효성 검증 및 인증요청	/cons/pccc/v1.0/verify/url	주문에 대한 관세청 유효성 검증 후 인증/접유확인을 위한 URL 응답 또는 간소화 응답하는 API	관세청의 PER001 대체 서비스
SMARTPCC_11	개인통관고유부호 유효성 검증 결과 및 인증 결과 확인	/cons/pccc/v1.0/verify/result	주문 정보에 대한 관세청 인증번호/1회용통관부호 등 결과 응답하는 API	관세청의 PER001 대체 서비스
SMARTPCC_20	주문 및 수하인 정보 수정	/cons/pccc/v1.0/verify/info	개인통관부호유효성 재검증 결과를 받기 위한 API	관세청의 PER002 대체 서비스
SMARTPCC_30	(사후처리)개인통관고유부호 유효성 검증 및 인증번호 발급	/cons/pccc/v1.0/verify/post	일시 중단 기간 내에 발생한 주문건에 대해, 서비스 재개 후 사후처리를 제공하는 API	관세청의 PER003 대체 서비스

1.5 인증팝업 요청

NICE 관세청 통관고유부호 인증 서비스 API 요청 시 인증 및 인가를 위해 아래 HTTP Header를 설정해야 합니다.

인증URL 요청 후, 응답받은 auth_url 을 Popup으로 호출하거나 수하인에게 알림톡/LMS 등으로 발송합니다.

```

<!-- popup -->
<script language='javascript'>
    function fnAuthNiceWeb(){
        window.open('https://smartpcc.niceid.co.kr/pccc/cert/request/S1afc40674-b094-44e7-be4b-3e42011b5f81'
, 'authNiceWeb', 'width=480, height=812, top=100, fullscreen=no, menubar=no, status=no, titlebar=yes,
location=no, toolbar=no, scrollbar=no');
    }
</script>
<html>
<body>
    <a href="javascript:fnAuthNiceWeb();">    </a>
</body>
</html>

```

- 별도 요청할 form이 없습니다. GET/POST로 요청 가능

- 인증팝업이 정상적으로 호출되고 사용자가 인증을 완료하면 **인증URL** 요청시 세팅한 **return_url** (https://your-domain.co.kr/return) 으로 result_inquiry_id값이 리턴됩니다.

- 트랜잭션의 경우 수하인 = 주문자 같을 경우 10분이며, 수하인 !=주문자 일 경우 24시간 동안 유효합니다.

다음은 인증 후 리턴되는 URL 예시입니다.

[GET 방식 예시]

https://your-domain.co.kr/return?result_inquiry_id=" ID"

- Method Type : GET
- GET 방식: URL에 포함된 result_inquiry_id값을 Query String에서 꺼내 사용합니다.

[POST 방식 예시]

- Method Type : POST
- POST 방식: Body에 담긴 result_inquiry_id값을 POST 데이터에서 꺼내 사용합니다.

- 해당 값을 받지 못할 경우 사용자 인증이 완료되지 않은 상태입니다.

- 해당 값을 받았을 경우 사용자 인증이 완료되었으므로 **결과요청**을 통해 인증결과 데이터를 받을 수 있습니다.

2. API 호출

2.1 API 정보

통관 스마트 검증 서비스 API는 아래 도메인을 통해 제공되므로 이용기관 서버에서 Outbound 방화벽 허용이 필요합니다.

[방화벽]

서비스 명	접속 도메인	IP	PORT	비고
통관 스마트 검증 서비스	smartpcc.niceid.co.kr	121.162.155.88	443	모든 WAS 구간에 대한 방화벽 오픈 필요
휴대폰 본인확인	nice.checkplus.co.kr	121.131.196.215	443	Client 환경이 폐쇄망일 경우 허용 필요 - 휴대폰 본인확인 팝업(Client)
아이핀	cert.vno.co.kr	203.234.219.124	443	Client 환경이 폐쇄망일 경우 허용 필요 - 아이핀 팝업(Client)
인증서(공동, 금융)	cert.niceid.co.kr	121.131.196.209	443	Client 환경이 폐쇄망일 경우 허용 필요 - 인증서(공동, 금융) 팝업(Client)

[공통]

HTTP Header	
Method Type	POST
Content-type	application/json
charset	UTF-8
Authorization	(필수) 아래 각 API별 Authorization 요청 규격 확인
X-Cons-TimeStamp	(필수) 시스템표준시(UTC)를 기준으로 한 밀리초(ms) 단위 데이터 JAVA) System.currentTimeMillis()
X-Cons-DevLang	(예시) Linux/JAVA

- X-Cons-DevLang의 경우 사용 하시는 운영체제 및 개발언어를 기입 해주시면 됩니다.

```
header= {
  "Content-type"      : "application/json",
  "Authorization"     : "Basic " + "ABCDE12345QWERTY=...", // API
  "X-Cons-timeStamp"  : "1780619075123", // (UTC) (ms)
  "X-Cons-DevLang"    : "Linux/JAVA" // /
}
```

2.2 (SMARTPCC_01) 접근토큰요청 API

- access token은 24시간 유효합니다.
- Base64UrlEncoding의 경우 Base64.getURLEncoder().withoutPadding() 사용이 필요합니다.

```
String Auth = clientId + ":" + clientSecret;
String Base64Auth = Base64.getURLEncoder().withoutPadding().encodeToString(Auth.getBytes());
```

(Method) POST

(URI) /cons/pccc/v1.0/auth/token

(Content-Type) application/json; charset=UTF-8

(Authorization) Basic base64({client_id:client_secret})

Request

항목	타입	길이	필수여부	암호화	설명
grant_type	String		Y	N	client_credentials 고정 (대소문자 구분 없음)
request_no	String	50	Y	N	요청고유번호 (최소 20, 최대 50)

Response

항목	타입	길이	필수여부	암호화	설명
request_no	String	50	Y	N	요청한 요청 고유번호 리턴
result_code	String	4	Y	N	결과코드 - 권한 에러 관련 ("1" + 에러코드 3자리) - 1001 : Authorization 헤더 오류 - 시스템 에러 관련 ("2" + 에러코드 3자리) - 서비스 에러 관련 ("3" + 에러코드 3자리) - 정상응답 ("0000")
result_msg	String	100	Y	N	결과메시지 ("정상")
access_token	String		Y	N	JWT 토큰
expires_in	long		Y	N	토큰 만료 시간 (Unix time)
token_type	String		Y	N	"Bearer" 고정
iterators	int		Y	N	암복호화시 키유도함수 내 iterationCount값 (3~100) (JWT 토큰 내 iterators 값)
ticket	String		Y	N	암복호화시 키유도함수 내 password 값

Sample Data

Request	{ "grant_type": "client_credentials", "request_no": "A1234567890123456789" }
Response	{ "result_code": "0000", "result_msg": "응답성공", "request_no": "A1234567890123456789", "access_token": "eyJhbGciOiJIUzUxMiJ9.eyJpYXQiOiJlOTUyODIsImV4cC.....MjExMTExliwiaXRlcmF0aWV9ucyl6MjMlbnNjb3BlIjpbblk0iLCJGliwiVSlkMiLCJlI19.~sDEGE=", "expires_in": 1762940529000, "token_type": "Bearer", "iterators": 66, "ticket": "UzEyMDI1MTExMzA5NTQ0MjI3NDI1Njc3QTM0RDBBRUZEMjI5MzUzQzEz" }

2.3 (SMARTPCC_10) 주문정보 관세청 유효성 검증 및 인증 URL

- 관세청 시스템 PER001 에 대응하는 API 입니다.
- 수하인/주문자에 대한 정보를 전송하고 거래 유형/관세청 유효성확인 결과에 따라 응답을 받습니다.
 - (1) 일반인증 성공 시 사용자 확인이 필요한 URL이 제공됩니다. (URL은 24시간 유효)
 - 인증 후 return_web_url로 result_inquiry_id 값이 콜백 됩니다.
 - (2) 간소화인증 성공 시 URL 없이 result_inquiry_id 값이 즉시 제공됩니다.
 - 요청 정보(성명/휴대폰번호)가 관세청에 등록된 정보와 다를경우 URL이 제공됩니다.
 - 해당 URL에서 사용자 본인확인 후 성공 시 올바른 정보(성명/휴대폰번호)를 상거래업체에 제공합니다.
- result_inquiry_id 값은 SMARTPCC_11 의 필수 값으로 활용됩니다.

*access_token은 **접근 토큰 발급 API (SAMRTPCC_01)** 응답 값 입니다.
※ 응답 받은 문자열 그대로 입력하여 사용합니다. base64UrlEncoding 처리 하지 않습니다. ※

(Method) POST						
(URI) /cons/pccc/v1.0/verify/url						
(Content-Type) application/json; charset=UTF-8						
(Authorization) Bearer {access_token} # /cons/pccc/v1.0/auth/token에서 받은 access_token값						
Request						
항목	서브항목	타입	길이	필수여부	암호화	설명
request_no		String	50	Y	N	이용기관 요청고유번호 • 최소 20byte 이상, 최대 50byte 내
svc_type		String	1	Y	N	상거래업체에서 사용가능한 인증방식 • A: 간소화인증 • B: 일반인증(SMS, EMAIL 점유확인) * 간소화 인증 시 주문자/수하인 정보가 다르거나 , 간소화 업체가 아닌 경우 일반인증방식으로 전환
web_data		Object		N	N	인증팝업 전환 시 필요 데이터 * 일반인증은 필수 값 * 간소화인증은 성명/휴대폰정보 불일치인 경우 인증팝업 URL 제공
web_data→ JSON Object						
	return_web_url	String	250	Y	N	인증팝업 인증 후 리턴할 회원사 URL • 최대 250byte 까지
	close_web_url	String	250	N	N	인증팝업에서 닫기 버튼 클릭 시 이동할 URL -회원사가 지정한 페이지로 이동 후 처리는 회원사에서 결정
	http_method_type	String	4	N	N	응답받을 method 지정 : GET/POST • default는 GET
	exp_mods	List<String>	100	N	N	closeButtonOn : 닫기 버튼 표기
pccc_enc_data		String		Y	Y	통관 요청 정보를 암호화 한 값
pccc_enc_data→ JSON Object						
	ord_info	JSON		Y	N	주문 정보
	cnsi_info	JSON		Y	N	수하인 정보
- ord_info → JSON Object						
	-- ordrr_fnm	String	150	Y	N	주문자 이름
	-- ordrr_mobileneno	String	12	N	N	주문자 휴대폰번호(- 제외) • 간소화 인증인 경우 필수
	-- ordrr_key	String	88	N	N	CI • 간소화 인증 요청시에는 필수
	-- ord_no	String	52	N	N	주문번호
	-- ord_prod_nm	String	200	Y	N	상품명

		• pccc_enc_data json 데이터 <pre> { "ord_info": { "ordrr_fnm":"홍길동", "ordrr_mobilenno":"01012341234", "ordrr_key":"AKEIDKFLEIFSDKFASDF.....", "ord_no":"123123", "ord_prod_nm":"테스트상품", "ord_prod_price":"10000", "ord_prod_class":"3305" }, "cnsi_info": { "cnsi_fnm":"홍길동", "cnsi_mobilenno":"01003250137", "cnsi_pers_ecm":"P202603000000", "cnsi_psno":"22100" } } </pre>
Response		• 응답 데이터(일반인증) <pre> { "result_code":"0000", "result_message":"정상처리", "request_no":"03d4279e-3387-4c64-a2ea-f8f3da6d1c32", "result_type":"1", "transaction_id":"UzFENjkwMjU5RUM0QzgyOTlwMjYwNDI3MTAzNzUzNTIxMzFCMjQ5RDQ", "auth_url":"https://smartpccc.niceid.co.kr/pccc/cert/request/123123123123sdahfjlqi3eivjahsu2q " } </pre> • 응답데이터(간소화인증) <pre> { "result_code":"0000", "result_message":"정상처리", "request_no":"03d4279e-3387-4c64-a2ea-f8f3da6d1c32", "result_type":"2", "transaction_id":"UzFENjkwMjU5RUM0QzgyOTlwMjYwNDI3MTAzNzUzNTIxMzFCMjQ5RDQ", "result_inquiry_id":"OTkyMjE0ZGltdDhmNS00ZDQ5LWlzNDMtN2Y1YjdmOGRlbnkz" } </pre>

2.4 (SMARTPCC_11) 개인통관고유부호 유효성 검증 결과 및 인증 결과 확인

- 관세청 시스템 PER001 에 대응하는 API 입니다.
- SMARTPCC_10를 통해 획득한 result_inquiry_id 값으로 통관에 필요한 정보들을 수신합니다. (개인통관고유부호, 인증번호)

*access_token은 접근 토큰 발급 API (SAMRTPCC_01) 응답 값 입니다.

※ 응답 받은 문자열 그대로 입력하여 사용합니다. base64UrlEncoding 처리 하지 않습니다. ※

(Method) POST						
(URI) /cons/pccc/v1.0/verify/result						
(Content-Type) application/json; charset=UTF-8						
(Authorization) Bearer {access_token} # /cons/pccc/v1.0/auth/token에서 받은 access_token값						
Request						
항목	서브항목	타입	길이	필수여부	암호화	설명
result_inquiry_id		String	100	Y	N	인증URL에서 인증이 성공 후 리턴되는 result_inquiry_id 또는 SMARTPCC_10 API (/cons/pccc/v1.0/verify/url) 에서 응답한 result_inquiry_id값
transaction_id		String	100	Y	N	SMARTPCC_10 API (/cons/pccc/v1.0/verify/url) 에서 응답한 transaction_id 값
request_no		String	50	Y	N	SMARTPCC_10 API (/cons/pccc/v1.0/verify/url) 에 요청한 request_no
Response						
항목	서브항목	타입	길이	필수여부	암호화	설명

result_code		String	40	Y	N	결과코드 - 권한 에러 관련 ("1" + 에러코드 3자리) - 1001 : Authorization 헤더 오류 - 시스템 에러 관련 ("2" + 에러코드 3자리) - 서비스 에러 관련 ("3" + 에러코드 3자리) - 정상응답 ("0000")
result_msg		String	100	Y	N	결과메시지 ("정상")
request_no		String	50	Y	N	요청시 받은 request_no
enc_data		String		Y	Y	JSON Object를 암호화한 값
integrity_value		String		Y	N	무결성값 (enc_data)
enc_data → JSON Object						
	svc_type	String	1	Y	N	최종 진행한 인증방식 (A: 간소화인증, B: 일반인증) - 일반업체가 간소화인증 요청을해도 SMS점유확인으로 진행됨 - 또는, SMS점유확인 요청을 해도 장애시 Email점유확인으로 진행됨
	auth_method	String	1	Y	N	- S : SMS 점유확인 - E : 이메일 점유확인 - P: 간소화 - N :비갱신자
	pers_ecm	String	13	Y	N	개인통관부호 or 1회용 개인통관부호
	otp	String	6	Y	N	일회용 인증번호
	cnsi_fnm	String	100	N	N	최종 유효성 검증에 성공한 수하인 성명
	cnsi_mobileno	String	12	N	N	최종 유효성 검증에 성공한 수하인 휴대폰번호
Sample Data						
Request	<pre>{ "result_inquiry_id": "ZGlxOGZkYjUtMjE4NC00MDZmLTkxZjgtM2ZhNjA0OTdiZTY2", "transaction_id": "UzE0MUQyNkFDOEQ3NzYyMDIwMjUxMTEzMTAwMjMzMzZM4OTQ5QUwMkU", "request_no": "A1234567890123456789" }</pre>					
Response	<pre>{ "result_code": "0000", "result_message": "정상처리", "request_no": "03d4279e-3387-4c64-a2ea-f8f3da6d1c32", "enc_data": "UzFENjkwMjU5RUM0QzgyOTIwMjYwNDI3.....UzNTIxMzFCMjQ5RDQ", "integrity_value": "neSKi1jaqTBd_uly-1-FYS-A7euPHPajR0HCODnC3H" }</pre> enc_data 복호화 JSON <pre>{ "svc_type": "A", "auth_method": "P", "pers_ecm": "P202604000000", "otp": "123456", "cnsi_fnm": "홍길동", "cnsi_mobileno": "01000000000" }</pre>					

2.5 (SMARTPCC_20) 주문정보 관세청 유효성 재검증

- 관세청 시스템 PER002 에 대응하는 API 입니다.
- SMARTPCC_11를 통해 처리 완료된 거래(transaction_id)에 대해 변경된 수하인 정보를 확인합니다.

*access_token은 접근 토큰 발급 API (SAMRTPCC_01) 응답 값 입니다

※ 응답 받은 문자열 그대로 입력하여 사용합니다. base64UrlEncoding 처리 하지 않습니다. ※

(Method) POST						
(URI) /cons/pccc/v1.0/verify/info						
(Content-Type) application/json; charset=UTF-8						
(Authorization) Bearer {access_token} # /cons/pccc/v1.0/auth/token에서 받은 access_token값						
Request						
항목	서브항목	타입	길이	필수여부	암호화	설명
transaction_id		String	100	Y	N	API /cons/pccc/v1.0/verify/uri에서 응답한 transaction_id 값

request_no		String	50	Y	N	이용기관 요청고유번호 • 최소 20byte 이상, 최대 50byte 내
pccc_enc_data		String		Y	Y	통관 요청 정보를 암호화 한 값
pccc_enc_data→ JSON Object						
	- cnsi_fnm	String	150	Y	N	수하인 성명
	- cnsi_mobilenno	String	12	Y	N	수하인휴대폰번호
	- cnsi_psno	String	6	Y	N	수하인 우편번호
Response						
항목	서브항목	타입	길이	필수여부	암호화	설명
result_code		String	4	Y	N	결과코드 • 권한 에러 관련 ("1" + 에러코드 3자리) - 1001 : Authorization 헤더 오류 • 시스템 에러 관련 ("2" + 에러코드 3자리) • 서비스 에러 관련 ("3" + 에러코드 3자리) • 정상응답 ("0000")
result_msg		String	100	Y	N	결과메시지 ("정상")
request_no		String	50	Y	N	요청한 request_no 그대로 리턴
Sample Data						
Request		<pre>{ "request_no":"03d4279e-3387-4c64-a2eaf8f3da6d1c32", "svc_type":"B", "timestamp":1777253873489, "pccc_enc_data":"RlBUeDc3czdRZ2hEWmxHRPh1BP6alkC0ginlDHHjHJiDf4klx..... sk5kCKi0WL6XAsDn1va6kU25PbpHNv7Wil_g", "web_data": { "return_web_url":"https://회원사도메인/pccc/result", "close_web_url":"https://회원사도메인/pccc/close", "http_method_type":"post" } }</pre>				
		pccc_enc_data json 데이터 <pre>{ "cnsi_fnm":"홍길동", "cnsi_mobilenno":"01003250137", "cnsi_psno":"22100" }</pre>				
Response		<pre>{ "result_code":"0000", "result_message":"정상처리", "request_no":"03d4279e-3387-4c64-a2ea-f8f3da6d1c32", }</pre>				

2.6 (SMARTPCC_30) 개인통관고유부호 유효성 검증 및 인증번호 발급

- 관세청 시스템 PER003 에 대응하는 API 입니다.
- 관세청 장애/시스템 작업 등으로 사용할 수 없을 때 NICE 에서는 **0008 오류코드**를 응답합니다.
- 해당 시점 거래들의 경우 주문완료 처리를 진행하고, 관세청 시스템 복구 후 사후 검증 요청 시 본 API를 활용하여 인증번호를 사후 수령해주세요.

*access_token은 **접근 토큰 발급 API (SAMRTPCC_01)** 응답 값 입니다

※ 응답 받은 문자열 그대로 입력하여 사용합니다. base64UrlEncoding 처리 하지 않습니다. ※

(Method) POST						
(URI) /cons/pccc/v1.0/verify/post						
(Content-Type) application/json; charset=UTF-8						
(Authorization) Bearer {access_token} # /cons/pccc/v1.0/auth/token에서 받은 access_token값						
Request						
항목	서브항목	타입	길이	필수여부	암호화	설명

request_no		String	50	Y	N	이용기관 요청고유번호 • 최소 20byte 이상, 최대 50byte 내
svc_type		String	1	Y	N	상거래업체에서 사용가능한 인증방식 • A: 간소화인증 • B: 일반인증(SMS, EMAIL 점유확인) * 간소화 인증 시 주문자/수하인 정보가 다르거나, 간소화 업체가 아닌 경우 일반인증방식으로 전환
pccc_enc_data		String		N	Y	통관 요청 정보를 암호화 한 값
pccc_enc_data → JSON Object						
	ord_info	JSON		N	N	주문 정보
	cnsi_info	JSON		Y	N	수하인 정보
	- ord_info → JSON Object					
	-- ordrr_fnm	String	150	Y	N	주문자 이름
	-- ordrr_mobileno	String	12	N	N	주문자 휴대폰번호(- 제외) * 간소화 인증인 경우 필수 값
	-- ordrr_key	String	107	N	N	CI
	-- ord_no	String	52	<u>Y</u>	N	주문번호 ▪ 사후 확인 시에는 주문번호 필수
	-- ord_prod_nm	String	200	Y	N	상품명
	-- ord_prod_price	String	20	Y	N	상품가격
	-- ord_prod_class	String	10	Y	N	상품분류
	-- ord_cmpl_dttm	String	14	Y	N	주문 완료 일시
	-- call_dttm	String	14	Y	N	호출일시
	- cnsi_info → JSON Object					
	-- cnsi_fnm	String	150	Y	N	수하인 성명
	-- cnsi_mobileno	String	12	Y	N	수하인휴대폰번호
	-- cnsi_pers_ecm	String	13	N	N	* 일반인증(SMS/EMAIL)인 경우 필수 수하인 개인통관 고유 부호
	-- cnsi_psno	String	6	Y	N	수하인 우편번호
Response						
항목	서브항목	타입	길이	필수여부	암호화	설명
result_code		String	40	Y	N	결과코드 • 권한 에러 관련 ("1" + 에러코드 3자리) - 1001 : Authorization 헤더 오류 • 시스템 에러 관련 ("2" + 에러코드 3자리) • 서비스 에러 관련 ("3" + 에러코드 3자리) • 관세청 통관 서비스 결과 관련 ("0" + 결과코드 3자리) • 정상응답 ("0000")
result_msg		String	100	Y	N	결과메시지 ("정상")
request_no		String	50	Y	N	요청시 받은 request_no
enc_data		String		Y	Y	JSON Object를 암호화한 값
integrity_value		String		Y	N	무결성값 (enc_data)
enc_data → JSON Object						
	pers_ecm	String	13	Y	N	개인통관부호 or 1회용 개인통관부호
	otp	String	6	Y	N	일회용 인증번호

3. 중요 정보 보안

NICE에서는 HTTP Header 내 access_token 을 참고하여 암호화 하며

요청정보 검증 및 인증 URL(SMARTPCC_10 - /cons/pccc/v1.0/verify/url) API에 포함된 req_info에 대한 암호화와 결과 요청(/cons/pccc/v1.0/verify/result) API 요청에 포함된 enc_data 암호화 하고, 무결성 검증값을 생성합니다.

API 코드	API명	이용기 관 암호화	NICE 암호 화	무결성 값 제공	API URI
SMARTPCC_01	접근토큰요청	X	X	X	/cons/pccc/v1.0/auth/token
SMARTPCC_10공	개인통관고유부호 유효성 검증 및 인증요청	O	X	X	/cons/pccc/v1.0/verify/url
SMARTPCC_11	개인통관고유부호 유효성 검증 결과 및 인증 결 과 확인	X	O	O	/cons/pccc/v1.0/verify/result
SMARTPCC_20	주문 및 수하인 정보 수정	O	X	X	/cons/pccc/v1.0/verify/info
SMARTPCC_30	(사후처리)개인통관고유부호 유효성 검증 및 인 증번호 발급	O	O	O	/cons/pccc/v1.0/verify/post

이용기관에서 무결성 검증 및 데이터 암호화를 수행하기 위해서는 무결성키(Integrity Key), 대칭키(Symmetric Key), 초기벡터(Iv) 값이 필요합니다.

3.1 키 구성 요소

[키 유도 함수(kdfValue) 생성]

```
String keyString = kdfValue(ticket, transactionId, iterators);
public String kdfValue(String ticket, String transactionId, int iterators) {
    try {
        // 대칭키를 유도하기 위한 Spec
        PBESpec spec = new PBESpec(ticket.toCharArray(), transactionId.getBytes(), iterators, 512);
        SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
        // Base64 Encoding은 UrlEncoder 사용 및 withoutPadding() 적용
        return Base64.getUrlEncoder().withoutPadding().encodeToString(skf.generateSecret(spec).getEncoded());
    } catch (Exception e) {
        log.error("[NICE] key String 생성 중 오류 : {}", e.getMessage());
        throw new RuntimeException("Exception : " + e.getMessage());
    }
}
```

[KEY, HMAC]

무결성키, 대칭키는 키 유도함수(KDF)를 통해 키 자료에서 각각 추출하여 사용합니다.
키 유도함수 KDF는 PBKDF2(PBESpec 기반) 방식이며,

-키 길이 : 512bit
-입력값(input)은

1. 접근 토큰 발급 요청 API에서 리턴 받은 iterators, ticket
2. 인증 URL 요청 API에서 리턴 받은 transaction_id입니다.

- 키 유도 함수(KDF)를 통해 생성된 문자열의 앞에서부터 32byte를 대칭키로 사용합니다.

- 키 유도 함수(KDF)를 통해 생성된 문자열의 48번째 부터 32byte를 무결성 키로 사용합니다.

```
// 생성된 값은 Base64 디코딩하지 않고 문자열 그대로 사용합니다.
// 대칭키
byte[] keyBytes = keyString.substring(0, 32).getBytes();

// 무결성 키
String hmacKey = keyString.substring(48, 80);
```

[초기벡터(Iv)]

- 결과 요청 API에서 리턴되는 암호화 데이터(enc_data) 문자열의 앞에서부터 16byte를 Iv로 사용합니다.

해당 값은 복호화 로직에 포함 되어 있습니다.

```
// 생성된 값은 Base64 디코딩하지 않고 문자열 그대로 사용합니다.
// 해당 값은 복호화 로직에 포함 되어 있습니다.
// cipherEnc = enc_data
byte[] iv = Arrays.copyOfRange(cipherEnc, 0, 16);
```

3.2 결과 무결성 검증 가이드

인증 결과 요청(/ido/intc/{version}/auth/result) API를 통해 인증 결과를 받았을 경우, integrity_value 값이 일치하는지 무결성값 검증이 필요합니다.

1. 무결성값 검증을 위해 인증 결과 요청 API의 응답 데이터 중 enc_data에 대해 무결성 값을 생성 후 리턴 받은 integrity_value와 일치하는지 확인이 필요합니다.
2. 무결성 값은 정상 응답일 때 제공됩니다.

```
String value = enc_data;

public String getSha256MacBase64Value(String value, String hmacKey) {
    try {
        // hmacKey를 이용해 Hashed Mac값 추출
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(hmacKey.getBytes(), "HmacSHA256"));
        byte[] hashValue = mac.doFinal(value.getBytes());
        // Base64 Encoding은 UrlEncoder 사용 및 withoutPadding() 적용
        return Base64.getEncoder().withoutPadding().encodeToString(hashValue);
    } catch (Exception e) {
        log.error("[NICE] HMAC 생성 오류: {}", e.getMessage());
        throw new RuntimeException("Exception : " + e.getMessage());
    }
}
```

3.3 중요 정보 암호화 예시

구분	대상	
암호화	/cons/pccc/v1.0/verify/url /cons/pccc/v1.0/verify/post	- PBKDF2를 이용하여 대칭키(Key) 유도 - AES/GCM/NoPadding으로 데이터 암호화 <pre>// 대칭키(Key) 유도 String keyString = getKeyValue(/cons/pccc/v1.0/auth/token에서 받은 ticket, 요청 시 보내는 timestamp, /cons/pccc/v1.0/auth/token에서 받은 iterators); // 유도된 문자열에서 암호화 키(32byte) 추출 byte[] keyBytes = keyString.substring(0, 32).getBytes(); // 암호화 SecretKey secureKey = new SecretKeySpec(유도된 문자열에서 암호화 키(32byte), "AES"); Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding"); GCMParameterSpec gcmParameterSpec = new GCMParameterSpec(128, iv); cipher.init(1, secureKey, gcmParameterSpec); byte[] encrypted = cipher.doFinal(data); return Base64.getEncoder().withoutPadding().encodeToString(KdfUtil.concat(new byte[][]{iv, encrypted}));</pre>
암호화	/cons/pccc/v1.0/verify/info	- PBKDF2를 이용하여 대칭키(Key) 유도 - AES/GCM/NoPadding으로 데이터 암호화 <pre>// 대칭키(Key) 유도 String keyString = getKeyValue(/cons/pccc/v1.0/auth/token에서 받은 ticket, /cons/pccc/v1.0/verify/url에서 받은 transaction_id, /cons/pccc/v1.0/auth/token에서 받은 iterators); // 유도된 문자열에서 암호화 키(32byte) 추출 byte[] keyBytes = keyString.substring(0, 32).getBytes(); // 암호화 SecretKey secureKey = new SecretKeySpec(유도된 문자열에서 암호화 키(32byte), "AES"); Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding"); GCMParameterSpec gcmParameterSpec = new GCMParameterSpec(128, iv); cipher.init(1, secureKey, gcmParameterSpec); byte[] encrypted = cipher.doFinal(data); return Base64.getEncoder().withoutPadding().encodeToString(KdfUtil.concat(new byte[][]{iv, encrypted}));</pre>

복호화	/cons/pccc/v1.0/verify/result /cons/pccc/v1.0/verify/post /cons/pccc/v1.0/issue/customskey/result	- PBKDF2를 이용하여 대칭키(Key)와 HMAC키 유도 - HMAC-SHA256으로 무결성 검증 (데이터 위변조 방지) - AES/GCM/NoPadding으로 데이터 복호화 <pre>// 대칭키(Key) 유도 String keyString = getKeyValue(/cons/pccc/v1.0/auth/token에서 받은 ticket, /cons/pccc/v1.0/verify/url에서 받은 transaction_id, /cons/pccc/v1.0/auth/token에서 받은 iterators); // 유도된 문자열에서 암호화 키(32byte), HMAC키 추출 byte[] keyBytes = keyString.substring(0, 32).getBytes(); String hmacKey = keyString.substring(48, 80); // 무결성비교데이터 생성 Mac mac = Mac.getInstance("HmacSHA256"); mac.init(new SecretKeySpec(유도된 문자열에서 HMAC키 byte, "HmacSHA256")); byte[] hashValue = mac.doFinal(encData.getBytes()); return Base64.getUrlEncoder().withoutPadding().encodeToString(hashValue); * integrity_value 일치여부 비교 // 복호화 byte[] decodedData = Base64.getUrlDecoder().decode(enc_data); byte[] iv = Arrays.copyOfRange(decodedData, 0, 16); // GCM 모드에서는 IV가 암호화 데이터의 앞부분에 위치함 byte[] cipherText = Arrays.copyOfRange(decodedData, 16, decodedData.length); SecretKey secureKey = new SecretKeySpec(유도된 문자열에서 암호화 키(32byte), "AES"); Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding"); GCMParameterSpec spec = new GCMParameterSpec(128, iv); // GCM 인증 태그 길이는 128비트 고정 cipher.init(Cipher.DECRYPT_MODE, secureKey, spec); return new String(cipher.doFinal(cipherText), "UTF-8");</pre>
키유도함수		<pre>public String getKeyValue(String ticket, String transactionId, int iterators) { try { // 대칭키를 유도하기 위한 Spec PBEKeySpec spec = new PBEKeySpec(ticket.toCharArray(), transactionId.getBytes(), iterators, 512); SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256"); // Base64 Encoding은 UrlEncoder 사용 및 withoutPadding() 적용 return Base64.getUrlEncoder().withoutPadding().encodeToString(skf.generateSecret(spec).getEncoded()); } catch (Exception e) { log.error("[NICE] key String 생성 중 오류 : {}", e.getMessage()); throw new RuntimeException("Exception : " + e.getMessage()); } }</pre>

4. 인증 결과

4.1 공통응답코드

응답코드 분류	http Status	오류 형태	조치내용
0XXX	200	API 서비스 정상 응답	정상 또는 관세청 결과
1XXX	400 or 500	API Gateway 검증 처리 중 오류	'1014' 응답 제외 사용자 요청 및 설정 확인 필요
2XXX	500	API Gateway에서 API 서비스 연동처리 중 오류	NICE에 오류 확인 요청 필요
3XXX	200	API 서비스 처리 중 오류	각 오류에 따른 조치

4.2 API 서비스 상세 응답코드

항목	응답코드	비고	설명
API_RESULT_OK	0000	HttpStatus.OK	응답 성공
API_RESULT_FAIL	0001	HttpStatus.OK	수하인정보 불일치로 실패(성명/휴대폰번호), 본인인증 인증 후 재시도 가능
API_RESULT_FAIL	0002	HttpStatus.OK	수하인정보 불일치로 실패(성명/휴대폰번호/우편번호 등)
API_RESULT_FAIL	0003	HttpStatus.OK	존재하지 않거나 해지/정지/만료된 개인통관고유부호
API_RESULT_FAIL	0004	HttpStatus.OK	간소화 이용불가 업체(협력인정업체 아님)
API_RESULT_FAIL	0005	HttpStatus.OK	간소화 이용불가 업체(주문자/수하인 불일치)

API_RESULT_FAIL	0006	HttpStatus.OK	전자상거래부호 오류
API_RESULT_FAIL	0007	HttpStatus.OK	관세청 통신 오류
API_RESULT_FAIL	0008	HttpStatus.OK	관세청 시스템 점검(사후 처리)
API_RESULT_FAIL	0009	HttpStatus.OK	관세청 정보 API 오류
GW_AUTHORIZATION_HEADER_ERROR	1001	HttpStatus.BAD_REQUEST	Authorization header 오류
GW_INVALID_PARAMETER	1002	HttpStatus.BAD_REQUEST	파라미터 유효성 오류
GW_TOKEN_EXPIRATION_ERROR	1003	HttpStatus.BAD_REQUEST	접근 토큰 만료 오류
GW_TOKEN_VALIDATION_ERROR	1004	HttpStatus.BAD_REQUEST	유효한 접근 토큰 아님
GW_INVALID_CLIENT_ERROR	1006	HttpStatus.BAD_REQUEST	ClientID 권한 없음
GW_ACCESS_IP_DENIED	1007	HttpStatus.BAD_REQUEST	허용되지 않은 IP 접근
GW_FLOW_OVER_DENIED	1008	HttpStatus.BAD_REQUEST	유량 초과 오류
GW_NOT_ALLOWED_API	1009	HttpStatus.BAD_REQUEST	API 이용권한 없음
GW_URI_NOTFOUND	1010	HttpStatus.BAD_REQUEST	없는 URI 요청
GW_REQUEST_CONFIRM	1011	HttpStatus.BAD_REQUEST	요청 정보 확인 필요
GW_TOKEN_SCOPE_ERROR	1012	HttpStatus.BAD_REQUEST	토큰 내 Scope 유효하지 않음
GW_TOKEN_ETC_ERROR	1013	HttpStatus.BAD_REQUEST	토큰 검증 기타 오류
GW_INTERNAL_DATA_ERROR	1014	HttpStatus.INTERNAL_SERVER_ERROR	내부 데이터 처리중 오류
GW_INTERNAL_SERVER_ERROR	2001	HttpStatus.INTERNAL_SERVER_ERROR	내부 서버 처리중 오류
GW_INTERNAL_NETWORK_ERROR	2002	HttpStatus.INTERNAL_SERVER_ERROR	내부 구간 통신 오류
GW_INTERNAL_SERVICE_ERROR	2003	HttpStatus.INTERNAL_SERVER_ERROR	내부 서비스 처리 오류
GW_UNKNOWN_ERROR	2999	HttpStatus.INTERNAL_SERVER_ERROR	알 수 없는 오류
API_UNEXPECTED_MSG_FORMAT	3001	HttpStatus.OK	JSON 형식의 메시지가 아님
API_BASE64_DECODE_ERROR	3002	HttpStatus.OK	Base64 Decode Error
API_AUTHORIZATION_HEADER_ERROR	3003	HttpStatus.OK	Authorization Header 오류
API_INTERNAL_ERROR	3011	HttpStatus.OK	내부 로직 처리 오류
API_TIMEOUT_ERROR	3012	HttpStatus.OK	유효시간 초과 오류
API_CONNECT_ERROR	3013	HttpStatus.OK	내·외부 시스템 연결 오류
API_SYSTEM_ERROR	3014	HttpStatus.OK	내·외부 시스템 오류
API_INVALID_DATA	3021	HttpStatus.OK	유효하지 않은 항목 데이터
API_REQUEST_URI_ERROR	3022	HttpStatus.OK	유효하지 않은 URI
API_PARAMETER_NULL	3023	HttpStatus.OK	요청파라미터가 NULL
API_INSUFFICIENT_ARGS	3024	HttpStatus.OK	불충분한 파라미터 또는 데이터 개수
API_DATA_DECRYPT_ERROR	3025	HttpStatus.OK	암호화 데이터 복호화 오류
API_DATA_ENCRYPT_ERROR	3026	HttpStatus.OK	데이터 암호화 오류
API_INTEGRITY_CHECK_ERROR	3027	HttpStatus.OK	무결성 체크값 오류
API_DATA_DUP_ERROR	3028	HttpStatus.OK	중복데이터 존재
API_DATA_NOT_FOUND	3029	HttpStatus.OK	요청한 정보를 찾을 수 없음
API_INVALID_SCOPE	3031	HttpStatus.OK	유효하지 않은 서비스 Scope
API_NOT_CONFIRMED_TRAN	3032	HttpStatus.OK	성공건이 아니거나 유효시간 초과
API_ONCE_COMMITTED_TRAN	3033	HttpStatus.OK	기 결과 제공 건
API_TOKEN_MANDATORY_ERROR	3041	HttpStatus.OK	토큰에 필수 claim 없음
API_TOKEN_NOT_ISSUED	3042	HttpStatus.OK	나이스에서 발급한 토큰 아님
API_TOKEN_EXPIRATION_ERROR	3043	HttpStatus.OK	유효시간 만료된 토큰
API_TOKEN_INVALID_SCOPE	3044	HttpStatus.OK	토큰내 scope 유효하지 않음
API_TOKEN_INVALID_SIGN	3045	HttpStatus.OK	유효하지 않은 서명값
API_TOKEN_INVALID_ETC	3046	HttpStatus.OK	유효하지 않은 토큰

